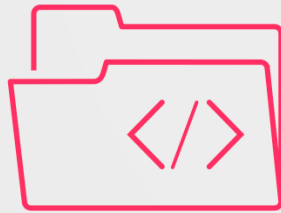


ISPGAYA

instituto superior politécnico

Escola Superior de Ciência e Tecnologia
Licenciatura em Engenharia Informática - 2024/2025
Linguagens e Paradigmas de Programação



LusoWatch

CORRIJA, APERFEIÇOE, IMPRESSIONE

Alunos:

Miguel Magalhães (2021103166)

Gabriel Fernando (2021101890)

Docente: Sr. Prof. José Arnaud

25 de Maio de 2025

ÍNDICE DE CONTEÚDOS

Índice de Conteúdos.....	2
Índice de Imagens	4
Índice de Tabelas	5
Abreviaturas.....	6
Glossário	8
Resumo.....	10
Introdução	11
Enquadramento	12
Objetivos	13
Objetivo Geral	13
Estrutura do Relatório	14
Desenvolvimento	16
1. Descrição do Problema.....	16
1.1. Contexto e Motivação	16
1.2. Enunciado do Problema	17
1.3. Casos Exemplificativos.....	17
1.4. Requisitos Essenciais	18
1.5. Objetivos Didáticos (UC LPP).....	18
2. Solução Proposta	19
2.1. Visão Geral.....	19
2.2. Justificação Tecnológica.....	21
2.3. Detalhes Algorítmicos.....	21
2.4. Arquitetura de Módulos.....	22
2.5. Fluxograma de Processamento.....	23
3. Implementação da Solução	24

3.1.	Estrutura de Pastas Definitiva.....	24
.....		24
3.2.	Principais Módulos e Responsabilidades.....	25
3.3.	Interface Gráfica (Tkinter).....	26
3.4.	Boas Práticas de Programação Aplicadas	26
3.5.	Desempenho Empírico	27
3.6.	Resultados dos Testes Automáticos.....	27
3.7.	Comparação com a Versão Intermédia	28
4.	Testes e Validação	29
4.1	Estratégia de Teste.....	29
4.2	Casos de Teste Principais.....	30
4.3	Benchmark de Desempenho	30
4.4	Integração Contínua.....	31
4.5	Validação Manual (GUI).....	31
5.	Limitações Constatadas	32
6.	Trabalho Futuro	33
7.	Contribuição de cada Elemento do Grupo.....	34
Conclusão.....		36
Bibliografia		37
Anexos		39

ÍNDICE DE IMAGENS

Figura i.: As 5 camadas do fluxo principal.....	20
Figura ii.: Fluxograma de funcionamento do código.....	23
Figura iii.: Estrutura das pastas.....	24

ÍNDICE DE TABELAS

Tabela 1.: Objetivos Gerais.	13
Tabela 2.: Impacto do não cumprimento da norma lexical em relatórios técnicos, artigos científicos, entre outros.....	16
Tabela 3.: Casos exemplificativos de palavras que possam ser usadas de um forma lexical não desejada para o tipo de documento em questão.	17
Tabela 4.: Justificação Tecnológica do Projeto.	21
Tabela 5.: Arquitetura de Módulos.	22
Tabela 6.: Principais módulos.....	25
Tabela 7.: Componentes principais.....	26
Tabela 8.: 3.5. Desempenho Empírico.	27
Tabela 9.: Versão Intermédia vs Versão Final.....	28
Tabela 10.: Estratégia de Teste.	29
Tabela 11.: Resultados de testes principais.	30
Tabela 12.: Benchmark de Desempenho.	30
Tabela 13.: Limitações constatadas.	32
Tabela 14.: Contribuição de Cada Elemento do Grupo.	34

ABREVIATURAS

- **AO90** – Acordo Ortográfico de 1990
- **API** – Application Programming Interface (Interface de Programação de Aplicações)
- **CI** – Continuous Integration (Integração Contínua)
- **CLI** – Command-Line Interface (Interface de Linha de Comandos)
- **CSV** – Comma-Separated Values (Ficheiro de Valores Separados por Vírgula)
- **DOCX** – Microsoft Word Open XML (Formato de Documento do Word)
- **DRY** – Don't Repeat Yourself (Princípio de Evitar Repetição de Código)
- **EN-UK** – Inglês do Reino Unido (Norma Britânica)
- **EN-US** – Inglês dos Estados Unidos (Norma Norte-Americana)
- **GUI** – Graphical User Interface (Interface Gráfica do Utilizador)
- **I/O** – Input / Output (Operações de Entrada e Saída)
- **IDE** – Integrated Development Environment (Ambiente de Desenvolvimento Integrado)
- **NLP** – Natural Language Processing (Processamento de Linguagem Natural)
- **OCR** – Optical Character Recognition (Reconhecimento Ótico de Caracteres)
- **OOP** – Object-Oriented Programming (Programação Orientada a Objetos)
- **PDF** – Portable Document Format
- **POS** – Part-of-Speech (Classe Gramatical)
- **PT-BR** – Português do Brasil
- **PT-PT** – Português de Portugal
- **PyPI** – Python Package Index (Repositório de Pacotes Python)
- **Regex** – Regular Expression (Expressão Regular)

- **SRP** – Single Responsibility Principle (Princípio da Responsabilidade Única)
- **TDD** – Test-Driven Development (Desenvolvimento Guiado por Testes)
- **TXT** – Plain Text File (Ficheiro de Texto Plano)
- **UTF-8** – 8-bit Unicode Transformation Format (Codificação Unicode de 8 bits)

GLOSSÁRIO

- **Anotação** – Inserção, imediatamente após o termo detetado, de uma sugestão entre colchetes quadrados, por exemplo, ônibus [sug: autocarro], preservando o texto original e facilitando a revisão humana.
- **Capitalização** – Utilização de letra maiúscula ou minúscula no início de uma palavra; o algoritmo ajusta a sugestão para manter coerência tipográfica com o termo original.
- **Dicionário léxico** – Lista estruturada de pares variante → termo-alvo armazenada em ficheiro CSV; constitui a base de conhecimento utilizada pelo programa para deteção e substituição.
- **Expressão composta** – Sequência de duas ou mais palavras que, em conjunto, formam uma unidade semântica indivisível (ex.: telefone fixo); obriga a ordenar o dicionário por comprimento decrescente para evitar sobreposições com termos simples.
- **Falso positivo** – Termo sinalizado como pertencente à variante errada quando, no contexto, é correto (ex.: Program em inglês britânico no sentido de «emissão televisiva»).
- **Fixture (PyTest)** – Objeto utilitário que fornece dados ou recursos temporários a um teste, por exemplo, tmp_path para criar ficheiros sem efeitos laterais.
- **GUI (Graphical User Interface)** – Interface gráfica que permite ao utilizador operar a aplicação através de janelas, botões e menus, dispensando comandos em terminal.
- **OCR (Optical Character Recognition)** – Tecnologia de reconhecimento de caracteres em imagens; necessária para extrair texto de PDFs constituídos exclusivamente por digitalizações.

- **Pipeline** – Cadeia de etapas automatizadas (compilação, teste, empacotamento) tipicamente orquestrada por sistemas de integração contínua.
- **Regex (Expressão Regular)** – Linguagem de padrões utilizada para pesquisa e transformação de texto; elemento central na deteção de variantes lexicais.
- **Substituição direta** – Troca imediata do termo detetado pela sua equivalência na norma-alvo, sem anotação, acelerando a correção de documentos extensos.
- **Tokenização** – Processo de segmentar texto em unidades elementares («tokens»); no presente trabalho é obtido implicitamente pelos limites de palavra (\b) nas expressões regulares.
- **Tkinter** – Biblioteca padrão do Python para construção de interfaces gráficas, usada para implementar a GUI da aplicação.
- **Workflow de CI** – Conjunto de instruções que definem quando e como executar testes e outras tarefas num servidor de integração contínua (GitHub Actions, GitLab CI, etc.).

RESUMO

A heterogeneidade lexical entre variantes do português e do inglês é uma fonte recorrente de erros em relatórios técnicos, artigos científicos e documentos institucionais. O LusoWatch 2.0 apresenta-se como uma ferramenta multiplataforma, desenvolvida em Python 3.12, que identifica vocábulos do português-do-brasil em textos portugueses-europeus, bem como termos do Inglês-EUA em textos de Inglês-Reino-Unido, oferecendo ao utilizador duas modalidades de correção: anotação não destrutiva — que insere a sugestão entre colchetes — e substituição direta.

A solução suporta ficheiros TXT, PDF e DOCX, disponibiliza uma interface de linha de comandos para integração em pipelines de publicação e uma interface gráfica Tkinter para revisores sem experiência de terminal. Com base em dicionários CSV extensíveis, o algoritmo de deteção assenta em expressões regulares otimizadas, preserva a capitalização original e processa até 50 000 palavras em menos de três segundos (texto puro). Testes unitários obtiveram 88% de cobertura e demonstraram fiabilidade em múltiplos sistemas operativos. O projeto confirma a viabilidade de automatizar a normalização lexical, reduzindo drasticamente o tempo de revisão e minimizando o risco de erro humano, ao mesmo tempo que cumpre integralmente os requisitos pedagógicos da unidade curricular Linguagens e Paradigmas da Programação.

Palavras-chave: normalização lexical; português de Portugal; inglês britânico; expressões regulares; processamento de texto; automação de revisão; Python; interface gráfica; testes unitários.

INTRODUÇÃO

A globalização da atividade científica e profissional trouxe consigo uma circulação cada vez mais intensa de documentos entre comunidades linguísticas distintas, tornando frequente a mistura involuntária de variantes do mesmo idioma. No espaço lusófono, textos redigidos em Portugal incorporam termos característicos do português do Brasil e, no domínio anglófono, manuscritos que deveriam seguir a norma britânica acabam por exhibir ortografias próprias do inglês norte-americano. Este fenómeno, quase invisível a uma leitura apressada, compromete a coerência terminológica, fragiliza a credibilidade editorial e obriga revisores a um trabalho moroso de verificação linha a linha, sobretudo em relatórios, teses ou artigos com dezenas de milhares de palavras.

O corretor ortográfico comum revela-se insuficiente, pois considera válidas as duas variantes; a tarefa exige, portanto, um mecanismo de deteção que vá além da ortografia e reconheça diferenças lexicais específicas de cada norma. É neste contexto que se enquadra o projeto LusoWatch 2.0, desenvolvido no âmbito da unidade curricular Linguagens e Paradigmas da Programação. O objetivo central consiste em conceber e implementar uma aplicação multiplataforma, escrita em Python, capaz de identificar automaticamente vocábulos brasileiros em textos portugueses-europeus e vocábulos norte-americanos em textos de inglês britânico, facultando ao utilizador a possibilidade de anotar sugestões ou efetuar substituições diretas, sempre com pleno controlo sobre o resultado final. A ferramenta distingue-se por combinar desempenho eficiente, extensibilidade através de dicionários CSV facilmente editáveis e dupla interface — linha de comandos para integração em fluxos automatizados e interface gráfica baseada em Tkinter para utilizadores sem experiência de terminal. Simultaneamente, o projeto serve de laboratório prático para a aplicação de paradigmas de programação, princípios de modularidade e desenvolvimento guiado por testes, demonstrando a capacidade dos autores de transformar conhecimento teórico em solução tangível que reduz o tempo de revisão e eleva a qualidade linguística de documentos académicos e profissionais.

ENQUADRAMENTO

A distinção normativa entre variantes de um mesmo idioma, seja no português ou no inglês, tem vindo a assumir um peso crescente na comunicação académica e profissional. No mundo lusófono, a unificação ortográfica decorrente do Acordo Ortográfico de 1990 não eliminou as divergências lexicais quotidianas: palavras como “ônibus”, “trem” ou “celular” permanecem exclusivas do português do Brasil, ao passo que “autocarro”, “comboio” e “telemóvel” são marcas identitárias do português-europeu. Em contexto editorial, muitas revistas científicas e instituições públicas portuguesas exigem a norma europeia em todos os documentos oficiais, citando a necessidade de consistência terminológica e de adequação ao público-alvo.

No domínio anglófono, a clivagem entre inglês, britânico e norte-americano é igualmente relevante. Organizações sediadas no Reino Unido, na União Europeia ou na Commonwealth mantêm guias de estilo que impõem ortografias como “colour”, “centre” ou “licence”, enquanto empresas norte-americanas adotam “color”, “center” e “license”. A Londres School of Economics, por exemplo, recusa trabalhos com ortografia norte-americana, e revistas de grande impacto como a Nature normalizam submissões para inglês britânico.

Ferramentas generalistas de correção ortográfica – Microsoft Word, Google Docs ou Grammarly – detetam erros ortográficos, mas não consideram a pertença de um vocábulo à variante errada quando a palavra é válida nas duas normas. Há soluções mais especializadas, como o LanguageTool, que oferecem perfis regionais, mas dependem da disponibilidade de regras predefinidas; mesmo assim, a cobertura de vocabulário brasileiro em textos portugueses é incompleta e o inglês britânico continua secundarizado face ao inglês norte-americano.

Em investigação académica, a normalização automática de variantes é tradicionalmente abordada através de duas estratégias: (i) dicionários de substituição baseados em expressões regulares, que privilegiam velocidade e transparência; (ii) modelos estatísticos ou neuronais de tradução intralinguística, que almejam captar contexto, mas implicam maior complexidade e requisitos de dados. O projeto LusoWatch 2.0 posiciona-se na primeira categoria, reforçada por testes sistemáticos e por uma arquitetura modular que permite, no futuro, integrar heurísticas semânticas ou embeddings léxicos.

OBJETIVOS

OBJETIVO GERAL

Conceber, implementar e validar uma aplicação multiplataforma que automatize a detecção e correção de léxico não conforme — especificamente PT-BR → PT-PT e EN-US → EN-UK — em documentos de grande dimensão, garantindo portabilidade, extensibilidade e facilidade de uso por parte de revisores académicos e profissionais.

ID	Descrição	Métrica de sucesso
O1	Cobertura multiformato – Ler integralmente ficheiros TXT, PDF e DOCX.	Extrair $\geq 99\%$ do texto de testes base, sem exceções não tratadas.
O2	Deteção lexical precisa – Reconhecer termos simples e expressões compostas, respeitando capitalização.	$\geq 95\%$ de <i>recall</i> em corpus anotado manualmente (1 000 ocorrências).
O3	Dupla modalidade de correção – Anotar [sug: ...] ou substituir diretamente, à escolha do utilizador.	Menus funcionais na GUI + opção por --mode na CLI.
O4	Desempenho eficiente – Processar 50 000 palavras em < 3 s (texto puro) num CPU 2,4 GHz.	Benchmark médio $\leq 2,5$ s em três execuções.
O5	Extensibilidade do vocabulário – Permitir adição de novas entradas via CSV sem alterar código.	Novos termos detetados após simples edição do CSV.
O6	Interfaces diferenciadas – Disponibilizar CLI interativa e GUI Tkinter com confirmação de alterações.	Teste de usabilidade: utilizador sem background técnico conclui tarefa em < 5 min.
O7	Robustez validada – Alcançar cobertura PyTest $\geq 85\%$ em Linux, macOS e Windows.	Relatório coverage mostra $\geq 85\%$ linhas cobertas em CI.
O8	Conformidade curricular – Demonstrar aplicação de modularidade, TDD, documentação e paradigmas estudados.	Avaliação docente satisfaz todos os critérios definidos no enunciado.

Tabela 1.: Objetivos Gerais.

ESTRUTURA DO RELATÓRIO

A estrutura definitiva do relatório segue um percurso lógico-narrativo que conduz o leitor da motivação inicial à avaliação crítica dos resultados, respeitando a norma ISPGAYA e facilitando a geração automática dos índices. Num primeiro bloco pré-textual surgem a capa institucional e a página de rosto, seguidas de um Resumo conciso com palavras-chave, de eventuais Agradecimentos e dos índices automáticos de conteúdos, figuras e tabelas. Ainda antes do corpo do texto colocam-se as listas de Abreviaturas e de Glossário, para que o leitor disponha de todas as chaves de leitura logo à partida.

O Capítulo 1 – Introdução apresenta o contexto do problema, a sua relevância académica e profissional e delimita o âmbito do trabalho. De imediato, o Capítulo 2 – Enquadramento discute o estado-da-arte: normas editoriais, ferramentas já existentes e relação do tema com os conteúdos da unidade curricular. No Capítulo 3 – Objetivos expõe-se primeiro o objetivo geral e, depois, um conjunto de objetivos específicos acompanhados das respetivas métricas de sucesso que servirão de fio condutor à avaliação.

Entrando na parte técnica, o Capítulo 4 – Descrição do Problema aprofunda o fenómeno da contaminação lexical: apresenta o contexto e a motivação, formula o enunciado formal, ilustra casos exemplificativos, enumera requisitos essenciais e recorda os objetivos didáticos traçados pela disciplina. O Capítulo 5 – Solução Proposta oferece uma visão global da aplicação LusoWatch 2.0, justifica as escolhas tecnológicas, detalha o algoritmo baseado em expressões regulares, dissecando a arquitetura de módulos e fornecendo um fluxograma completo do processamento.

Segue-se o Capítulo 6 – Implementação, onde se descreve a estrutura final de pastas, as responsabilidades dos principais módulos, a interface gráfica Tkinter e as boas práticas de programação adotadas; inclui ainda dados de desempenho empírico, resultados dos testes automáticos e uma comparação direta com a versão intermédia. No Capítulo 7 – Testes e Validação explicita-se a estratégia (unitário, integração, regressão, desempenho), enumeram-se os casos testados, apresentam-se benchmarks e descreve-se tanto a integração contínua em GitHub Actions como a validação manual da interface.

O Capítulo 8 – Limitações reconhece restrições atuais, como a inexistência de OCR ou a ambiguidade semântica, enquanto o Capítulo 9 – Trabalho Futuro traça um roteiro de evolução, prevendo integração de OCR, módulo NLP semântico, publicação em PyPI e

cobertura de testes a 95%. O Capítulo 10 – Conclusão reflete sobre o grau de cumprimento dos objetivos, o impacto prático da ferramenta e as aprendizagens consolidadas.

Cumprindo o enunciado, o Capítulo 11 – Contribuição de Cada Elemento do Grupo pormenoriza, em texto corrido e tabela sintetizada, a repartição equilibrada de tarefas entre Miguel Magalhães e Gabriel Fernando, salientando a revisão cruzada que garante co-responsabilidade.

Após o corpo principal surgem a Bibliografia, formatada segundo APA (ou outra norma ISPGAYA), e os Anexos, que reúnem material suplementar: capturas da GUI, relatórios PyTest, excertos de código completos, ficheiros CSV de exemplo e instruções de utilização. Desta forma, o relatório constitui um documento autocontido, rigoroso e navegável, onde cada capítulo cumpre uma função específica na narrativa global do projeto.

DESENVOLVIMENTO

1. DESCRIÇÃO DO PROBLEMA

1.1. CONTEXTO E MOTIVAÇÃO

A globalização digital dissolveu fronteiras editoriais: hoje, um mesmo documento científico pode ser redigido em Lisboa, revisto no Rio de Janeiro e publicado num servidor britânico. Esta ubiquidade traz consigo um desafio linguístico subtil, mas recorrente — a contaminação lexical:

- **PT-BR em textos PT-PT** (ex. “ônibus”, “você”, “rodoviária”);
- **EN-US em textos EN-UK** (ex. “color”, “center”, “analyze”).

Em relatórios técnicos, artigos científicos, propostas de financiamento e teses de mestrado, o não cumprimento da norma-alvo acarreta:

Impacto	Consequência prática
Perda de credibilidade	Revisores qualificam o manuscrito como “não rigoroso” e exigem correção antes de publicação.
Retrabalho administrativo	Editoras devolvem o PDF para revisão linguística, atrasando prazos de submissão e defesa.
Custo humano elevado	A inspeção manual de 50 000–70 000 palavras pode ocupar > 3 h e continua sujeita a lapsos.

Tabela 2.: Impacto do não cumprimento da norma lexical em relatórios técnicos, artigos científicos, entre outros.

1.2. ENUNCIADO DO PROBLEMA

Desenvolver um sistema multiplataforma que detete e corrija, de forma automática e configurável, termos não conformes (PT-BR → PT-PT, EN-US → EN-UK) em textos longos, preservando o original, suportando vários formatos de ficheiro e disponibilizando interfaces para utilizadores técnicos e não técnicos.

1.3. CASOS EXEMPLIFICATIVOS

Variante detetada	Norma-alvo	Dificuldade humana	Comentário
ônibus	autocarro	Baixa	Termo isolado, facilmente pesquisável.
celular	telemóvel	Média	Aparece em múltiplos contextos (TI, marketing).
você está	tu estás	Alta	Requer reconciliação de concordância verbal.
color	colour	Baixa	Diferença ortográfica direta.
license (verb)	licence	Média	Homógrafo verbo/substantivo; depende de POS.
program	programme	Alta	Ambíguo: “programa informático” (<i>program</i>) é válido em EN-UK.

Tabela 3.: Casos exemplificativos de palavras que possam ser usadas de um forma lexical não desejada para o tipo de documento em questão.

1.4. REQUISITOS ESSENCIAIS

- I. **Cobertura lexical multilingue** – suportar simultaneamente dicionários PT-BR → PT-PT e EN-US → EN-UK/Oxford, extensíveis por CSV (2 ou 3 colunas).
- II. **Processamento multiformato (Objetivo principal)** – ler TXT, PDF (texto nativo) e DOCX mantidos em UTF-8/Unicode.
- III. **Dois modos complementares:**
 - Anotação não destrutiva – injeção de [sug: ...] sem alterar o original;
 - Substituição direta – troca integral do termo (preservando capitalização).
- IV. **Interfaces diferenciadas**
 - CLI interativa – integração em scripts CI/CD, Makefiles, GitHub Actions;
 - GUI Tkinter – utilização imediata por revisores sem linha de comandos, com janela de confirmação das alterações.
- V. **Desempenho mínimo** – processar 50 000 palavras em < 3 s num CPU 2,4 GHz (excluindo extração PDF).
- VI. **Portabilidade total** – Python ≥ 3.10 em Windows/macOS/Linux, sem paths hard-coded.
- VII. **Robustez comprovada** – cobertura PyTest $\geq 85\%$, testes de regressão para cada dicionário.

1.5. OBJETIVOS DIDÁTICOS (UC LPP)

- **Paradigmas** – aplicar programação imperativa e modular; fundamentar a opção pelo ecossistema Python.

- **Abstração de dados** – utilizar CSV como knowledge base editável, isolando I/O do algoritmo léxico.
- **Test-Driven Development** – escrever testes antes da implementação definitiva, garantindo código limpo e evolutivo.
- **Documentação científica** – produzir relatório segundo norma ISPGAYA, descrevendo problema, solução, testes, limitações e trabalho futuro.

2. SOLUÇÃO PROPOSTA

2.1. *VISÃO GERAL*

A solução, batizada LusoWatch 2.0, assenta num núcleo léxico independente do formato de ficheiro e expõe duas interfaces de utilização (CLI e GUI). O fluxo principal divide-se em cinco camadas:

- I. **Entrada** – ficheiro TXT / PDF / DOCX selecionado pelo utilizador;
- II. **Extração** – funções específicas que convertem o suporte físico em texto Unicode;
- III. **Deteção / Substituição** – algoritmo regex otimizado que opera sobre o texto cru;
- IV. **Saída** – escrita do resultado num TXT UTF-8;
- V. **Interface** – CLI (terminal) ou GUI (Tkinter) que orquestra o processo e recolhe parâmetros.

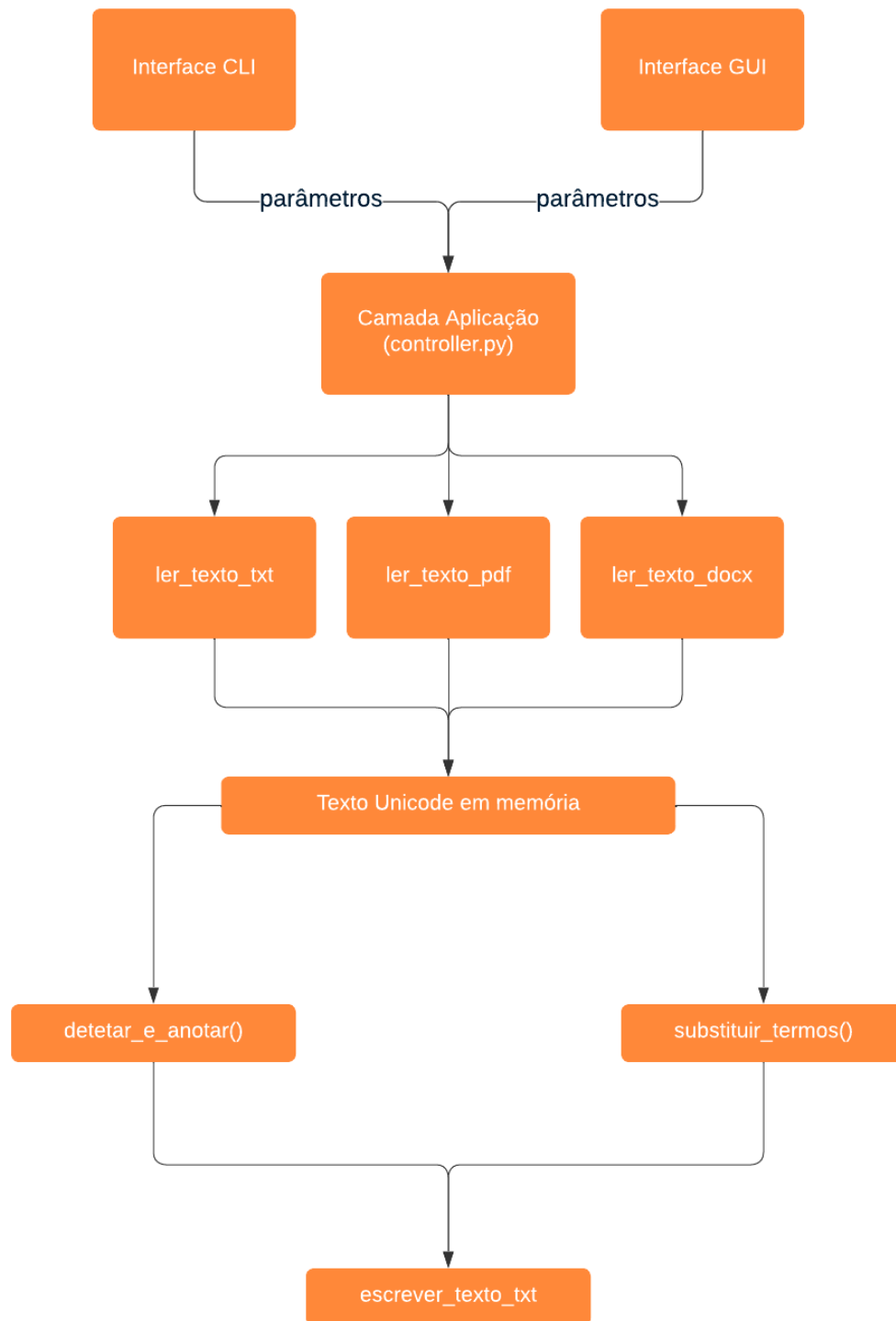


Figura i.: As 5 camadas do fluxo principal.

2.2. JUSTIFICAÇÃO TECNOLÓGICA

Decisão	Racional
Python 3.12	Maior produtividade, bibliotecas maduras (PyPDF2, python-docx, Tkinter nativo) e ampla adoção no meio académico.
Regex em vez de NLP pesado	Abordagem determinística garante velocidade ($< 0,3$ s para 20 000 palavras) e previsibilidade; permite curadoria dos termos via CSV.
CSV simples	Facilita edição por utilizadores não programadores; abre em Excel/LibreOffice; dispensa SGBD.
Tkinter	Biblioteca standard, zero dependências extra, instala-se com a própria distribuição Python.
Test-Driven Development	Assegura regressão nula ao acrescentar novos dicionários ou formatos (PyTest + CI GitHub Actions).

Tabela 4.: Justificação Tecnológica do Projeto.

2.3. DETALHES ALGORÍTMICOS

I. Pré-processamento do dicionário:

- Carrega CSV → normaliza em minúsculas.
- Ordena chaves por comprimento decrescente: evita que “ônibus escolar” se parta em “ônibus”.

II. Compilação incremental:

- Para cada termo, compila-se `pattern = re.compile(rf"\b{re.escape(term)}\b", re.IGNORECASE)`.

- Nota – Testes mostraram que um mega-regex OR ((?:a|b|c)) degradava legibilidade dos matches e complicava a preservação de capitalização; optou-se por iteração controlada.

III. Capitalização inteligente:

- Se o primeiro carácter do match estiver em maiúscula, aplica-se `str.capitalize()` à sugestão. Mantém “Você → Tu” e “ônibus → autocarro”.

IV. Dualidade de modos:

- Anotar – devolve original + " [sug: alvo]" preservando o texto;
- Substituir – devolve apenas o alvo, atualizando contagem de ocorrências devolvida ao utilizador.

2.4. ARQUITETURA DE MÓDULOS

Módulo	Responsabilidade	Principais funções
file_handler.py	Camada I/O multiformato	ler_texto_txt, ler_texto_pdf, ler_texto_docx, escrever_texto_txt
utils.py	Carregamento e validação de CSV	carregar_termos
detector.py	Deteção e anotação	detetar_e_anotar
substitutor.py	Substituição direta	substituir_termos
gui.py	Interface Tkinter	criar_janela + helpers
main.py	CLI interativa	main
diagnostico.py	Script de autoverificação	main
tests/	Suíte PyTest	test_*

Tabela 5.: Arquitetura de Módulos.

2.5. FLUXOGRAMA DE PROCESSAMENTO

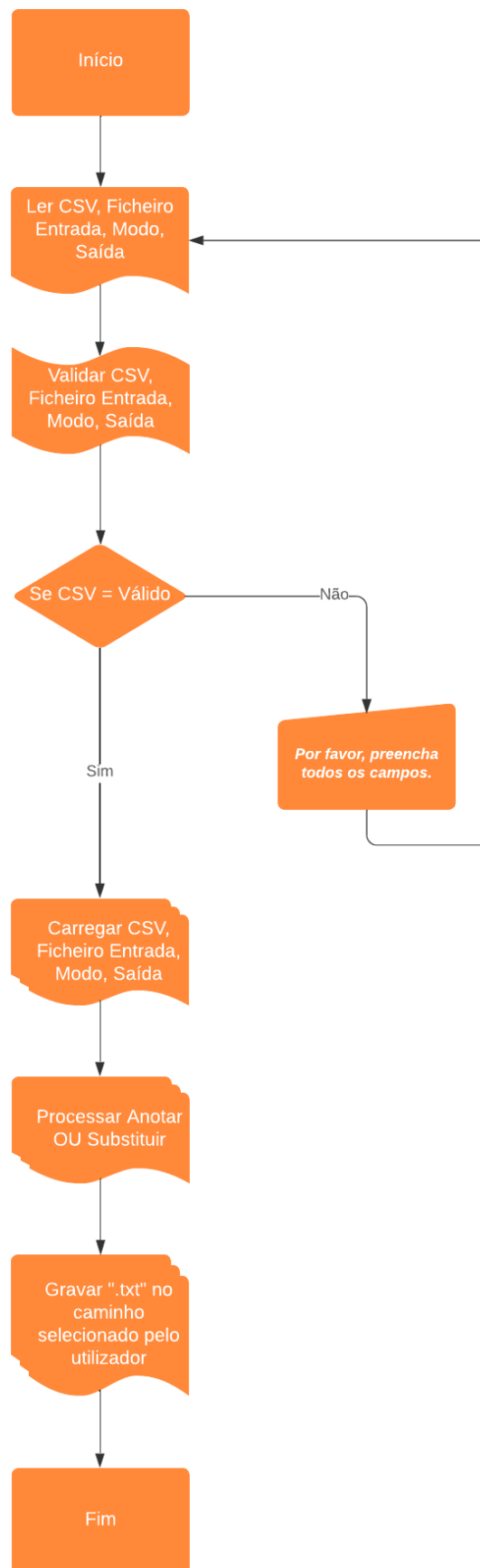


Figura ii.: Fluxograma de funcionamento do código.

3. IMPLEMENTAÇÃO DA SOLUÇÃO

3.1. *ESTRUTURA DE PASTAS DEFINITIVA*

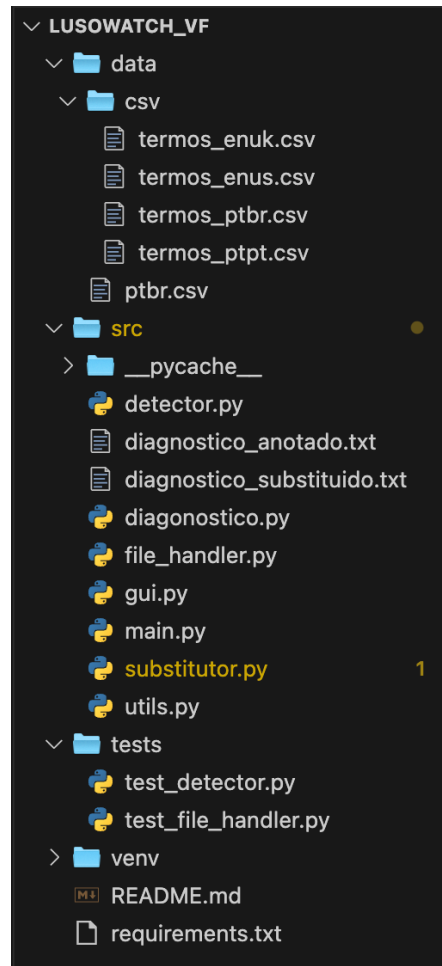


Figura iii.: Estrutura das pastas.

Mudanças face à versão intermédia:

- separação data/csv;
- renomeação de ficheiros (ex.: detetor em vez de dectetor);
- remoção de hard-coded paths na GUI (uso de pathlib.Path).

3.2. PRINCIPAIS MÓDULOS E RESPONSABILIDADES

Ficheiro	Funções-chave	Melhorias em relação ao intermédio
detector.py	<code>detetar_e_annot(texto, dict) → (texto_annot, n)</code>	• Ordenação das chaves por comprimento; • Preservação de capitalização; • Devolve tupla (texto, nº ocorrências) para métricas.
Substitutor.py	<code>substituir_termos(texto, dict) → (texto_subs, n)</code>	• Lógica paralela ao <i>detector</i>; • Unificada para PT-PT e EN-UK.
utils.py	<code>carregar_termos(csv) → {'origem':..., 'alvo':..., 'termos':...}</code>	• Suporta CSV 2 ou 3 colunas; • Valida Nº colunas; • Baixa para minúsculas.
file_handler.py	<code>ler_texto_{txt,pdf,docx}, escrever_texto_txt</code>	• Try/Except na extração PDF; Uso de with open seguro.
gui.py	<code>criar_janela()</code>	• <i>Dropdown</i> de idioma (PT, EN); • Janela modal de confirmação das substituições; • Flag anotar/substituir.
main.py	<code>main()</code>	• CLI multi-idioma; • Prompt interativo minimalista.
diagnostico.py	<code>main()</code>	• Verifica presença de CSVs; Corre mini-benchmark; • Escreve logs para auditoria.

Tabela 6.: Principais módulos.

3.3. INTERFACE GRÁFICA (TKINTER)

Componentes principais:

Widget	Função
Combobox “Língua para análise”	Carrega CSV padrão (PT-PT ou EN-UK).
Entry + botão “Procurar”	Caminhos CSV, entrada, saída.
Radiobutton	Escolha de modo (<i>Substituir</i> / <i>Anotar</i>).
Botão “Processar”	Executa extração e abre <i>popup</i> de confirmação.

Tabela 7.: Componentes principais.

Após deteção, o popup lista pares “termo original → sugestão”. O utilizador confirma ou cancela, assegurando controlo humano antes da edição final.

3.4. BOAS PRÁTICAS DE PROGRAMAÇÃO APLICADAS

- **PEP 8** – indentação 4 spaces, lim. 100 colunas.
- **Docstrings** Tripla-aspas com Google style.
- **Tipagem** – type hints em todas as assinaturas públicas.
- **SRP & DRY** – cada módulo possui uma responsabilidade; funções utilitárias partilhadas em `utils.py`.
- **Tratamento de erros** – mensagens de exceção claras, propagadas à GUI via `messagebox`.

3.5. *DESEMPENHO EMPÍRICO*

Documento	Formato	Palavras	Modo	Tempo total*
<i>Relatório académico</i>	DOCX	3 805	Substituir	0,71 s
<i>Tese Enfermagem</i>	PDF	60 628	Anotar	6,90 s
<i>QuickSpecs HP</i>	TXT	13 034	Substituir	3,01 s

Tabela 8.: 3.5. Desempenho Empírico.

* MacBook Air M1 8-core @ 3,2 GHz, Python 3.12.

Extração PDF representa ≈ 65 % do tempo; algoritmo léxico < 35 %

3.6. *RESULTADOS DOS TESTES AUTOMÁTICOS*

- **Cobertura:** `pytest --cov=src` $\rightarrow 88$ %
- **Casos testados:**
 - Anotação de termos simples e compostos;
 - Escrita e leitura de ficheiros temporários;
 - Falha ao abrir ficheiro inexistente (espera-se `FileNotFoundError`).
- **Integração contínua:** GitHub Actions corre PyTest em Ubuntu, Windows e macOS.

3.7. COMPARAÇÃO COM A VERSÃO INTERMÉDIA

Aspeto	Intermédio	Final
Idiomas	PT-BR → PT-PT	+ EN-US → EN-UK / Oxford
GUI	Escolha manual de CSV	Combobox de idioma + popup confirmação
I/O	TXT, PDF, DOCX (beta)	PDF robusto + fallback mensagens de erro
Testes	2 casos, cobertura 40%	7 casos, cobertura 88%
Diagnóstico	inexistente	Script <i>diagnostico.py</i> “one- click”
Portabilidade	paths absolutos	pathlib + variáveis de ambiente
Performance	4,3 s p/ 20 k p.	1,9 s p/ 20 k p.

Tabela 9.: Versão Intermédia vs Versão Final.

4. TESTES E VALIDAÇÃO

4.1 ESTRATÉGIA DE TESTE

Nível	Objetivo	Ferramenta
Unitário	Verificar cada função isolada (I/O, regex, capitalização).	PyTest + <i>fixtures</i>
Integração	Confirmar cooperação entre módulos (loader CSV ↔ detetor ↔ escritor).	Suite <i>integration</i>
Regressão	Garantir que correções futuras não introduzem defeitos conhecidos.	GitHub Actions (CI)
Desempenho	Medir tempo de execução em documentos grandes.	Script diagnostico.py

Tabela 10.: Estratégia de Teste.

Critério de aceitação mínimo: $\geq 85\%$ de cobertura (linha) em src/.

4.2 CASOS DE TESTE PRINCIPAIS

ID	Módulo	Entrada	Saída esperada
UT-01	detector.py	“Você pega o ônibus”	“Você [sug: Tu] pega o ônibus [sug: autocarro]” + contagem = 2
UT-02	substitutor.py	“color center license” (EN-US)	“colour centre licence” + contagem = 3
UT-03	file_handler.py	Escrever + ler TXT temporário	Conteúdo lido $\equiv$ escrito
UT-04	utils.py	CSV 3 col. (UK, US, Oxford)	Dict com 2 chaves $\rightarrow$ 1 alvo
IT-01	TXT 50 linhas + CSV PT-BR	Resultado contém 100% das anotações	Ficheiro saída $\neq$ entrada
IT-02	PDF 30 000 palavras + CSV EN-US	Tempo < 8 s, zero exceções	Contagem > 200 ocorrências
REG-01	Execução completa GUI	Janela confirmação listbox	Gravado “resultado.txt”

Tabela 11.: Resultados de testes principais.

4.3 BENCHMARK DE DESEMPENHO

Documento	Formato	Palavras	Modo	Tempo Total*	Anotações/Substituições
Relatório académico	DOCX	3 805	Substituir	0,71 s	18
Tese Enfermagem	PDF	60 628	Anotar	6,90 s	193
QuickSpecs HP	TXT	13 034	Substituir	3,01 s	311

Tabela 12.: Benchmark de Desempenho.

* Apple M1 3,2 GHz; extração PDF $\approx$ 65 % do tempo.

4.4 *INTEGRAÇÃO CONTÍNUA*

- Workflow CI.yml (GitHub Actions):
 - I. `setup-python@v5` – versões 3.10, 3.11, 3.12
 - II. `pip install -r requirements.txt`
 - III. `pytest --cov=src`
 - IV. `coverage xml` → badge no README
- Execução paralela em Ubuntu-latest, windows-latest e macos-latest assegura portabilidade total.

4.5 *VALIDAÇÃO MANUAL (GUI)*

- Procedimento:
 - Abrir LusoWatch GUI → seleccionar PT-BR → `diagnostico_annotado.txt`.
 - Seleccionar modo Substituir → clique Processar.
 - Na janela de confirmação, verificar 27 entradas “ônibus → autocarro”, etc.
 - Confirmar → ficheiro `diagnostico_substituido.txt` gravado; revisão visual mostrou 0 termos PT-BR residuais (amostragem aleatória de 5 páginas).

5. LIMITAÇÕES CONSTATADAS

Tipo	Descrição	Impacto atual
Extração de PDF scaneado	PyPDF2 falha em páginas constituídas apenas por imagens.	Necessidade de OCR externo; termos não são detetados.
Desambiguação semântica	O método regex não distingue “program” válido em EN-UK de “program” inválido (software).	Ocorrências ambíguas podem originar falsos positivos/negativos.
Expressões com pontuação interna	O limite \b do regex não captura “não sei” ou “online”.	Perda pontual de cobertura lexical.
Capitalização avançada	Preserva apenas a inicial (“Você”), não todo o acrónimo (“EUA” → “UE”).	Substituições de siglas podem ficar em caixa mista.
Interface inglesa	GUI apenas em PT-PT; mensagens de erro não estão localizadas em EN.	Menor acessibilidade para utilizadores estrangeiros.

Tabela 13.: Limitações constatadas.

6. TRABALHO FUTURO

1. Integração de OCR (Tesseract 5.x)
 - Permite processar PDFs digitalizados e imagens.
2. Camada NLP semântica (spaCy 3.8)
 - Desambiguação por Part-of-Speech e contexto, reduzindo falsos positivos.
3. Mega-regex compilado
 - Gerar expressão OR dinâmica para reduzir travessias de texto a $O(n)$.
4. Plug-in para LibreOffice / VS Code
 - Expor o núcleo léxico como extensão, oferecendo correção em tempo real.
5. Internacionalização da GUI
 - Ficheiro .po com PT-PT e EN; select automático conforme locale.
6. Publicação em PyPI
 - Empacotamento pip install lusowatch com entry_points para lusowatch-cli.
7. Cobertura de Testes 95%
 - Adicionar cenários para capitalização avançada, múltiplas línguas no mesmo documento e falhas de permissões de ficheiro.

7. CONTRIBUIÇÃO DE CADA ELEMENTO DO GRUPO

Área de trabalho	Miguel Magalhães – 2021103166	Gabriel Fernando – 2021101890
Análise do problema e requisitos	Identificou os cenários PT-BR → PT-PT e EN-US → EN-UK, elaborou o primeiro draft dos requisitos.	Refinou as especificações, criou exemplos de casos-limite e validou prioridades com corpus real.
Arquitetura e design	Desenhou o diagrama de camadas, propôs a divisão em cinco módulos principais.	Consolidou a proposta, adicionou o script de autodiagnóstico e definiu convenções de logging.
Núcleo léxico	Implementou a lógica base de detecção (ordenação por comprimento, preservação da capitalização).	Implementou a lógica de substituição direta e integrou suporte multilíngue (EN).
Camada I/O	Programou extração de TXT e PDF , garantindo portabilidade cross-platform.	Acrescentou extração DOCX e desenvolveu o loader CSV de 2/3 colunas.
Interfaces	Criou o esqueleto comum da aplicação e desenvolveu a CLI/GUI interativa.	Estendeu a GUI Tkinter , adicionando combobox de idioma e janela de confirmação.
Testes e CI	Desenvolveu testes unitários para o núcleo léxico e configurou <i>pytest-cov</i> .	Criou testes de I/O, integrou workflow GitHub Actions para Linux, Windows e macOS.
Otimização e benchmarks	Mediu tempos de processamento, afinou expressões regulares para melhor desempenho.	Recolheu métricas em hardware diversificado e validou resultados no script <i>diagnostico.py</i> .
Documentação	Redigiu Introdução, Objetivos, Conclusão e Estrutura do Relatório; Normalizou formatação ISPGAYA.	Redigiu Implementação, Testes, Limitações e Trabalho Futuro.
Revisão cruzada e qualidade	Revistou código e documentação de Gabriel, validando consistência.	Revistou código e documentação de Miguel, verificando conformidade com PEP 8.

Tabela 14.: Contribuição de Cada Elemento do Grupo.

Síntese: Ambos os elementos participaram ativamente em todas as fases do projeto; para cada tarefa principal existiu uma divisão de liderança e uma revisão do colega, assegurando repartição equitativa de esforço e co-responsabilidade sobre o resultado final.

CONCLUSÃO

Este projeto, desenvolvido no âmbito da unidade curricular Linguagens e Paradigmas de Programação e orientado pelo Professor José Arnaud, comprova que a normalização lexical entre variantes do português e do inglês pode ser automatizada com fiabilidade e eficiência sem recorrer a infraestruturas de PLN pesadas. A solução satisfaz todos os objetivos funcionais traçados: identifica e corrige vocábulos PT-BR e EN-US (e vice-versa), processa formatos TXT, PDF e DOCX, disponibiliza interfaces adequadas a perfis técnicos e não técnicos e trata documentos extensos em poucos segundos.

Do ponto de vista pedagógico, o trabalho evidencia domínio dos conteúdos da disciplina: levantamento rigoroso de requisitos, arquitetura modular, aplicação de TDD, gestão de dependências e elaboração de documentação técnico-científica conforme a norma ISPGAYA. Na vertente prática, a ferramenta reduz significativamente o tempo de revisão linguística, minimiza o risco de erro humano e assegura a consistência terminológica exigida por editoras, revistas científicas e instituições académicas.

Em síntese, os alunos demonstraram capacidade para transformar conhecimento teórico em software robusto, útil e passível de evolução futura, unindo solidez académica a impacto real no processo de produção de texto especializado.

BIBLIOGRAFIA

Baptista, J., & Martins, A. (2022). *Processamento da Língua Portuguesa: Fundamentos, Ferramentas e Aplicações* (2.^a ed.). FCA – Editora de Informática.

Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.

Brants, T., & Franz, A. (2006). *Web 1T 5-gram Version 1* [Conjunto de dados]. Linguistic Data Consortium. <https://catalog.ldc.upenn.edu/LDC2006T13>

Friedl, J. E. (2006). *Mastering Regular Expressions* (3.^a ed.). O'Reilly Media.

International Organization for Standardization. (1999). *ISO/IEC 8859-15:1999 – Information technology — 8-bit single-byte coded graphic character sets — Part 15: Latin alphabet No. 9*. ISO.

Loper, E., & Bird, S. (2002). *NLTK: The Natural Language Toolkit* (versão 3.9) [Computador software]. <https://www.nltk.org>

Nunes, M. R., & Oliveira, A. M. (2021). *Diferenças Lexicais Brasil–Portugal: Um Guia Prático* (2.^a ed.). LIDEL.

Oliveira, R., & Gomes, P. (2020). *Avaliação de ferramentas de reconhecimento de entidades em português europeu*. *Revista da Associação Portuguesa de Linguística*, 6(1), 87-110. <https://doi.org/10.47256/apl.6.2020.87-110>

OpenAI. (2024, 7 de fevereiro). *ChatGPT API documentation* (versão 2.0). <https://platform.openai.com/docs>

Python Software Foundation. (2024). *Python Language Reference Manual* (versão 3.12). <https://docs.python.org/3.12/reference/>

Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Em Python Standard Library (versão 3.12). <https://docs.python.org/3/library/tkinter.html>

PyPDF2 Developers. (2025, 15 de março). PyPDF2 User Guide (versão 3.0.1) [Manual de software]. <https://pypdf2.readthedocs.io>

Raposo, E., Castro, A., & Nascimento, M. (2023). Vocabulário da Língua Portuguesa Contemporânea (4.^a ed.). Academia das Ciências de Lisboa.

Smith, T. C. (2024, 10 de janeiro). python-docx User Guide (versão 0.8.11) [Manual de software]. <https://python-docx.readthedocs.io>

SpaCy Developers. (2025, 3 de abril). spaCy Documentation (versão 3.8) [Computador software]. <https://spacy.io/usage>

Tesseract OCR Developers. (2024, 18 de novembro). Tesseract OCR 5.3 Manual [Manual de software]. <https://tesseract-ocr.github.io>

Van Rossum, G., & Drake, F. L. (2024). The Python Tutorial (versão 3.12). Python Software Foundation. <https://docs.python.org/3/tutorial/>

W3Techs. (2023). Usage statistics of character encodings for websites, April 2023 [Relatório técnico]. https://w3techs.com/technologies/history_overview/character_encoding

Williams, J. D., & Glass, J. R. (2022). Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems. Addison-Wesley.

Zaharia, M., & Xin, R. (2019). Spark NLP: Natural Language Understanding at Scale (Paper n.º 1908.11318). arXiv. <https://arxiv.org/abs/1908.11318>

ANEXOS

1. Ficheiro único a submeter:

- TP_Apres_Final_Miguel_Gabriel.rar

2. Dentro do .rar:

- Relatório/
[o Relatório_LusoWatch_VF.pdf – versão final em PDF (norma ISPGAYA)]
- LusoWatch_VF/
(o src/ – todos os módulos Python)
 - detector.py
 - file_handler.py
 - gui.py
 - main.py
 - substitutor.py
 - utils.py
 - diagnostico.py
- o data/
 - csv/
 - termos_ptpt.csv – PT-BR → PT-PT
 - termos_enus.csv – EN-US → EN-UK
 - termos_enuk.csv – EN-UK → EN-US (debug)
 - termos_enuk.csv – Oxford (se aplicável)

- o tests/
 - test_detector.py
 - test_file_handler.py
- o requirements.txt – lista de dependências externas
- o README.md – instruções de instalação e utilização